Midterm Project Report

Name: Tong Khanh Ly
Student ID: 23071036
Email: 23071036@vnu.edu.vn

October 19, 2026

Abstract

This project implements three classical computer vision pipelines with Python and OpenCV, without deep learning. In Part A, a photo taken from my home is corrupted with Gaussian and salt-and-pepper noise and restored with mean, Gaussian and median filters and Laplacian sharpening. The median filter removes impulse noise best (35.9 dB), and the Gaussian filter is best for Gaussian noise (31.4 dB). In Part B, two stereo pairs from the Middlebury dataset are used to estimate the fundamental matrix with RANSAC, compute disparity with block matching (BM) and semi-global matching (SGBM), and build a rotatable 3D point cloud. SGBM is denser and more accurate than BM, but slower. In Part C, four overlapping photos of my neighbourhood are stitched using SIFT, ORB and AKAZE features, RANSAC homography, cylindrical warping and multi-band blending. AKAZE is the fastest, SIFT the most precise, and the result is a flat, seamless panorama.

1 Introduction

Deep networks are now the default tool in many vision tasks, but many of their building blocks come from classical image processing: convolution and smoothing, multi-view geometry and feature matching. These classical methods are still widely used, because they are fast, need no training data, and every step can be explained mathematically. The goal of this midterm project is to implement three such pipelines, measure them with quantitative metrics, and explain their behaviour.

- **Part A – Image filtering.** Real photos always contain noise from the sensor or from transmission. Denoising is a trade-off: removing noise also removes fine detail. We compare linear (mean, Gaussian) and non-linear (median) filters on two noise types, and use Laplacian sharpening to bring edges back.
- **Part B – 3D reconstruction.** With two calibrated views, depth can be recovered from disparity. This is the basis of depth cameras, robot navigation and 3D scanning. We estimate the epipolar geometry, compare a local (BM) and a semi-global (SGBM) stereo matcher against ground truth, and convert disparity to a 3D point cloud.
- **Part C – Image stitching.** Phone panoramas are built with local features, robust model fitting and blending. We compare three feature detectors and several RANSAC settings, and study how the choice of projection (planar or cylindrical) affects how flat the result is.

All code is written in Python with OpenCV [1], NumPy, Matplotlib, scikit-image (only for SSIM) and Plotly (interactive 3D). Every metric is a separate function in `metrics.py`. Runtimes are the median of 7 runs on the same CPU. Sections 2–4 follow the same struc-

ture: methodology, implementation and results, and comparative analysis. Section 5 concludes. Bonus items include self-captured photos for Parts A and C, a third detector (AKAZE), Laplacian-pyramid (multi-band) blending, a cylindrical projection, and sensitivity studies for kernel size, σ, blockSize, numDisparities and the RANSAC threshold.

2 Part A: Image Filtering

2.1 Methodology

Noise models. Let f be the clean grayscale image and $\tilde{f}$ the noisy one.

- *Gaussian noise* (sensor/thermal noise): $\tilde{f}(x,y) = f(x,y) + n(x,y)$ with $n \sim \mathcal{N}(0, \sigma_n^2)$, $\sigma_n = 20$.
- *Salt-and-pepper noise* (dead pixels, transmission errors): each pixel becomes 0 with probability $p/2$ and 255 with probability $p/2$, with $p = 5\%$.

Mean filter. Each output pixel is the average over a $k \times k$ window Ω:

$$g(x,y) = \frac{1}{k^2} \sum_{(i,j)\in\Omega} f(x+i,\, y+j). \tag{1}$$

It is linear and very fast. Its frequency response is a sinc, whose side lobes let some high-frequency noise through, and it blurs edges as much as it smooths flat regions.

Gaussian filter. The window is weighted by a 2D Gaussian, so closer pixels count more:

$$G_\sigma(x,y) = \frac{1}{2\pi\sigma^2} \exp\!\left(-\frac{x^2+y^2}{2\sigma^2}\right), \qquad g = G_\sigma * f. \tag{2}$$

The Gaussian is separable (two 1D passes), and its Fourier transform is again a Gaussian, so it has no side lobes and does not cause ringing. σ controls the amount of smoothing. For each kernel size, OpenCV's rule $\sigma = 0.3\,((k-1)/2-1)+0.8$ is used ($\sigma = 0.8$, 1.1, 1.4 for $k = 3, 5, 7$). In addition, $\sigma = 0.5$ and $\sigma = 2.0$ are tested at $k = 5$ to separate the effect of σ from the effect of k.

Median filter. The pixel is replaced by the median of its window, $g(x,y) = \text{median}\{f(x+i, y+j) : (i,j) \in \Omega\}$. The median is non-linear: an extreme value (0 or 255) in the window does not affect it as long as fewer than half of the window pixels are corrupted. This makes the median filter ideal for impulse noise, and it keeps step edges.

Laplacian sharpening. The Laplacian is a second derivative. With the 4-neighbour kernel:

$$\nabla^2 f = f(x{+}1,y) + f(x{-}1,y) + f(x,y{+}1) + f(x,y{-}1) - 4f(x,y), \qquad \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}. \tag{3}$$

It is large around edges and zero in flat regions. Sharpening subtracts it from the image, with strength α:

$$g = f - \alpha\, \nabla^2 f. \tag{4}$$

Because it amplifies high frequencies, it also amplifies noise. For this reason, it is applied either to a clean blurred image or after denoising ("Gaussian 5×5 + Laplacian", "Median 3×3 + Laplacian"). In both cases $\alpha = 0.5$.

Metrics. Let f be the reference image, g the result, and N the number of pixels.

- MSE $= \frac{1}{N}\sum(f-g)^2$, and PSNR $= 10\log_{10}\left(255^2/\text{MSE}\right)$ in dB (higher is better).
- SSIM $= \dfrac{(2\mu_f\mu_g + C_1)(2\sigma_{fg} + C_2)}{(\mu_f^2 + \mu_g^2 + C_1)(\sigma_f^2 + \sigma_g^2 + C_2)}$, which compares luminance, contrast and structure in local windows [5]. 1 means identical.
- Edge Preservation Index (EPI), the correlation between the Laplacian of f and that of g [6].
- Edge-F1, the F1 score between the Canny edges of f and of g (1 px tolerance). It measures whether the edges survive.
- Runtime in ms, the median of 7 repetitions.

2.2 Implementation and Results

The input image `IMG_3115.jpg` (Figure 1) was taken with my phone from the window at home. Its left half is a distant city view with many fine edges (buildings, trees, a road), and its right half is a nearby wall that is almost flat. This mix shows both how well noise is removed (on the wall and sky) and how well detail is kept (on the city). The image is converted to grayscale and resized to 800×450. The noise is generated with a fixed seed (42), so that the results can be reproduced.

The OpenCV functions are `cv2.blur` (mean), `cv2.GaussianBlur`, `cv2.medianBlur` and `cv2.Laplacian(ksize=1)`, which uses the kernel of Eq. (3). The image is converted to float before the Laplacian, so that negative values are not clipped. Each of the 14 filter settings is applied to both noisy images and evaluated against the clean image.

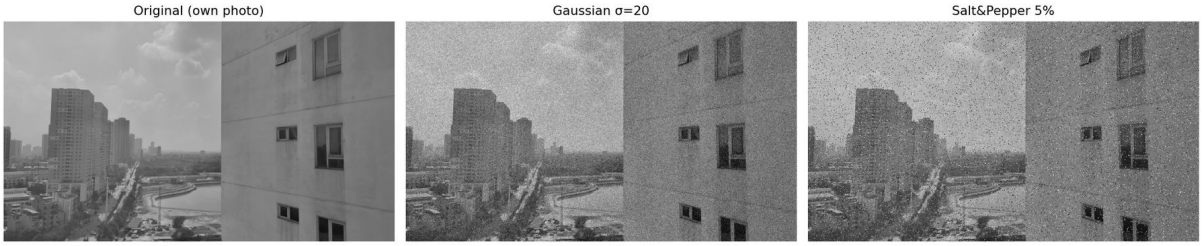


Figure 1: Part A input (own photo, 800×450) and the two noisy versions: Gaussian $\sigma_n = 20$ (22.11 dB) and salt-and-pepper 5% (18.76 dB).

Laplacian sharpening of a blurred image. To study sharpening on its own, the clean image is first blurred (Gaussian 5×5, $\sigma = 1.5$) and then sharpened with $\alpha = 0.5$, 1, 2 (Figure 5, Table 3). The variance of the Laplacian is used as a no-reference sharpness score.

2.3 Comparative Analysis

All metrics are listed in Tables 1–3, and the effect of the kernel size is shown in Figure 7.

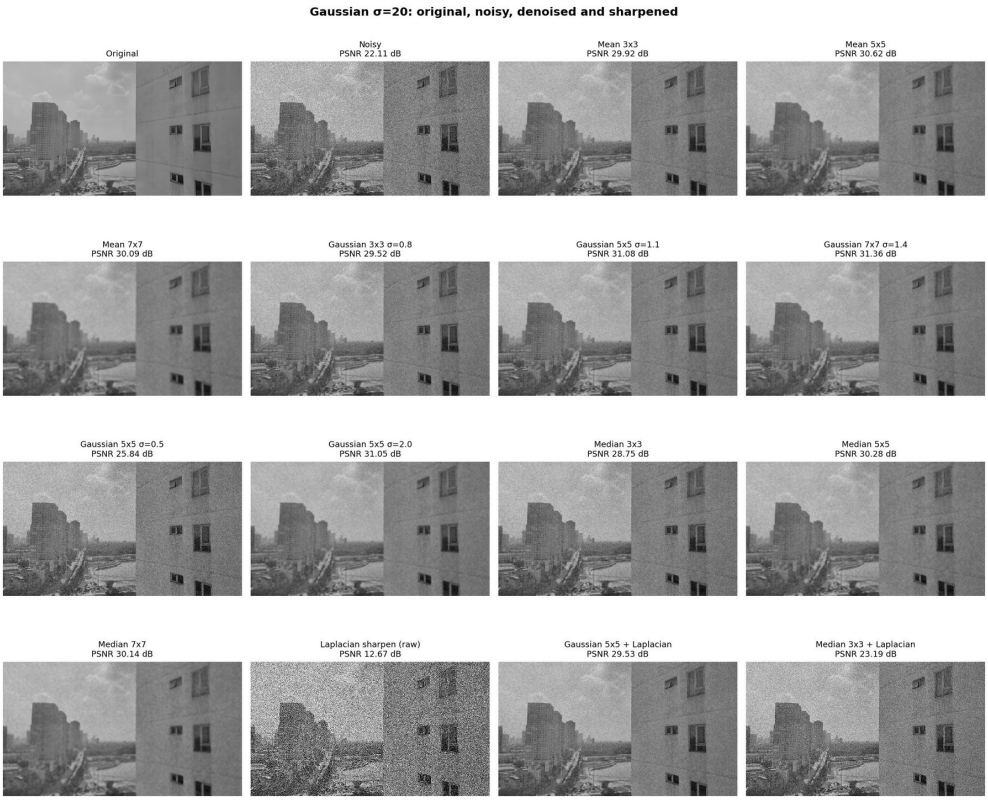


Figure 2: Gaussian noise: original, noisy, all denoised results and the sharpened results (PSNR in each title).

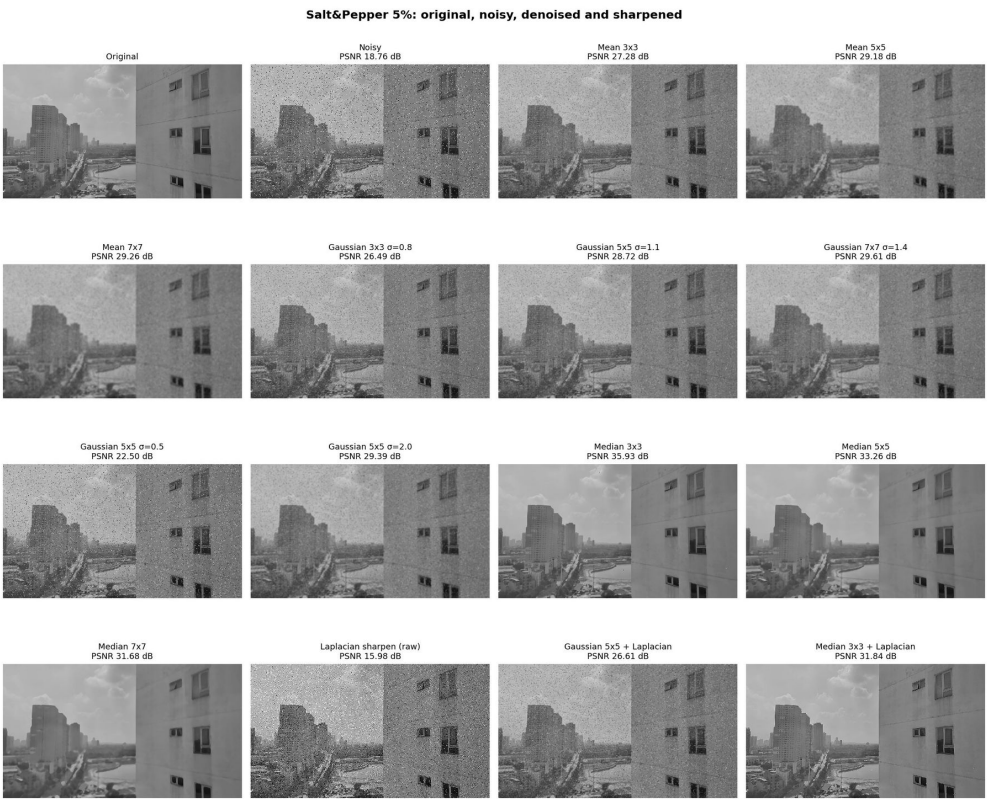
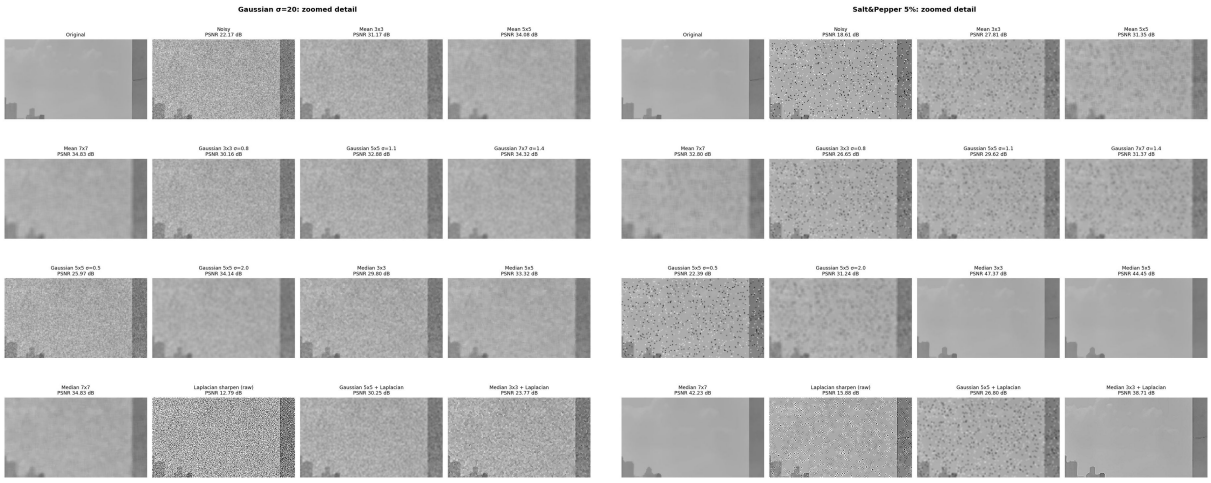


Figure 3: Salt-and-pepper noise: the same 14 settings. Only the median filters remove the impulses. The linear filters spread every white or black dot into a grey blob.



(a) Gaussian noise (b) Salt-and-pepper noise

Figure 4: Zoom on a detailed area (windows and building edges) to compare how well edges are kept.



Figure 5: Laplacian sharpening: original, blurred, and sharpened with $\alpha = 0.5, 1, 2$.

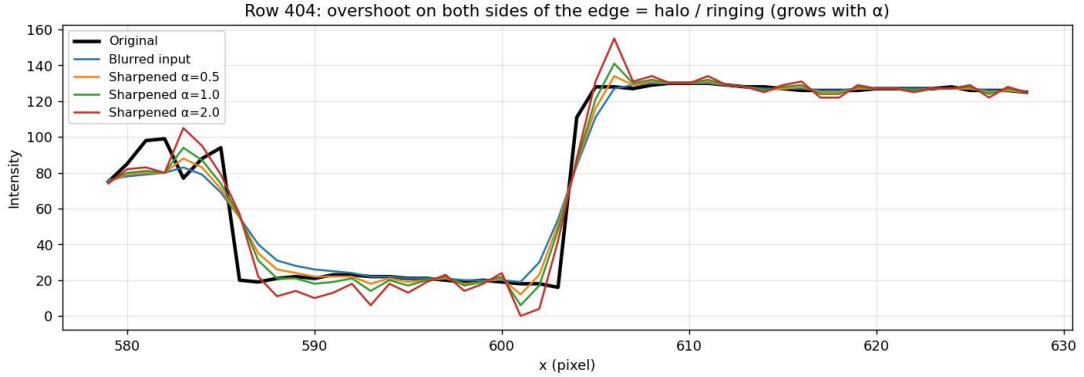


Figure 6: Intensity profile across an edge. Larger α makes the edge steeper, but it also creates an overshoot and an undershoot on each side of the edge (halo).

Noise removal and the matching filter.

- *Gaussian noise* affects every pixel by a small amount, so a weighted average is the right tool. Gaussian 7×7 (σ=1.4) gives the highest PSNR (31.36 dB, +9.3 dB) and the lowest MSE. The median filter is weaker here (28.75 dB at 3×3), because the median of Gaussian-distributed values is a noisier estimate than the mean.
- *Salt-and-pepper noise* affects few pixels, but by a very large amount. The median filter ignores these outliers: Median 3×3 reaches 35.93 dB and SSIM 0.954 (+17.2 dB), and it also has the best EPI and Edge-F1. Linear filters cannot remove an impulse; they only spread it. Even the best one, Gaussian 7×7, stays at 29.6 dB and leaves grey blobs (Figure 3).

Noise removal versus edge preservation (over-smoothing). A larger kernel removes more noise but also removes detail. The metrics show this clearly. For the mean filter on Gaussian noise, SSIM increases from 0.65 to 0.80 when going from 3×3 to 7×7, but Edge-F1 drops from 0.73 to 0.07: almost all Canny edges of the city disappear (Figures 4 and 7). SSIM is still high because most of the image is sky and wall, where heavy smoothing helps. So SSIM alone is not enough to judge a filter, and an edge metric is needed. The same happens with σ at a fixed 5×5 kernel. With $\sigma = 0.5$ the filter barely smooths (25.8 dB). With $\sigma = 2.0$ the PSNR is the same as with $\sigma = 1.1$ (31.05 against 31.08 dB), but EPI drops from 0.26 to 0.08, so the image is blurrier for the same noise reduction. For impulse noise, the best median is the smallest one: Median 7×7 also removes thin structures such as window frames (31.7 dB, Edge-F1 0.53).

Halo and ringing. Laplacian sharpening makes edges steeper, but it overshoots. In Figure 6, the profile goes above the bright side and below the dark side of each edge, which shows up as a bright or dark halo. With $\alpha = 2$, the sharpness score (259.9) is close to the original (253.4) and Edge-F1 is the highest (0.85), but PSNR and SSIM fall below the blurred input (33.25 dB, 0.881), because the halos are wrong values. $\alpha = 1$ is the best compromise (34.62 dB). On a noisy image, sharpening is harmful: "Laplacian (raw)" drops to 12.7 dB because the Laplacian amplifies the noise. The correct order is to denoise first and then sharpen. Median 3×3 + Laplacian on salt-and-pepper noise gives the best Edge-F1 of all settings (0.843) and keeps 31.8 dB. The Gaussian filter itself does not ring, because its spectrum has no negative lobes. The mean filter's sinc response does have side lobes, which is one reason why it leaves more residual high-frequency noise than the Gaussian filter at the same kernel size.

Runtime. All linear filters take less than 0.25 ms at 800×450, because they are separable or use an integral image. The median filter is fast for $k = 3$ and $k = 5$ (0.06–0.6 ms) but takes about 10.9 ms for $k = 7$. OpenCV switches to a slower histogram-based algorithm for $k \geq 7$, so Median 7×7 is more than 100 times slower than Median 3×3 and also gives worse quality.

Table 1: Part A metrics against the clean image (Gaussian noise $\sigma_n = 20$). The best value among the denoisers is in bold.

Filter	PSNR (dB)	SSIM	MSE	EPI	Edge-F1	Time (ms)
Noisy (no filter)	22.11	0.251	400.1	–	–	–
Mean 3×3	29.92	0.653	66.2	0.053	**0.727**	0.07
Mean 5×5	30.62	0.777	56.4	−0.008	0.358	0.09
Mean 7×7	30.09	**0.800**	63.7	−0.006	0.072	0.25
Gaussian 3×3, σ=0.8	29.52	0.609	72.6	**0.269**	0.465	0.10
Gaussian 5×5, σ=1.1	31.08	0.739	50.7	0.261	0.673	0.12
Gaussian 7×7, σ=1.4	**31.36**	0.796	**47.5**	0.232	0.495	0.22
Gaussian 5×5, σ=0.5	25.84	0.399	169.6	0.215	0.307	0.12
Gaussian 5×5, σ=2.0	31.05	0.786	51.1	0.079	0.451	0.14
Median 3×3	28.75	0.575	86.8	0.106	0.494	0.08
Median 5×5	30.28	0.731	61.0	0.068	0.562	0.59
Median 7×7	30.14	0.780	62.9	0.057	0.345	10.92
Laplacian sharpen (raw)	12.67	0.065	3518	0.121	0.297	1.03
Gaussian 5×5 + Laplacian	29.53	0.618	72.5	0.241	0.502	1.23
Median 3×3 + Laplacian	23.19	0.288	312.2	0.055	0.303	1.03

Table 2: Part A metrics against the clean image (salt-and-pepper noise, 5%).

Filter	PSNR (dB)	SSIM	MSE	EPI	Edge-F1	Time (ms)
Noisy (no filter)	18.76	0.244	865.9	–	–	–
Mean 3×3	27.28	0.532	121.5	0.034	0.414	0.08
Mean 5×5	29.18	0.691	78.5	−0.003	0.454	0.08
Mean 7×7	29.26	0.752	77.0	−0.004	0.062	0.22
Gaussian 3×3, σ=0.8	26.49	0.495	145.9	0.183	0.383	0.15
Gaussian 5×5, σ=1.1	28.72	0.628	87.3	0.186	0.521	0.14
Gaussian 7×7, σ=1.4	29.61	0.708	71.2	0.177	0.528	0.22
Gaussian 5×5, σ=0.5	22.50	0.341	365.4	0.143	0.362	0.12
Gaussian 5×5, σ=2.0	29.39	0.697	74.9	0.060	0.501	0.15
Median 3×3	**35.93**	**0.954**	**16.6**	**0.457**	**0.816**	0.06
Median 5×5	33.26	0.912	30.7	0.342	0.653	0.57
Median 7×7	31.68	0.878	44.2	0.290	0.535	10.65
Laplacian sharpen (raw)	15.98	0.162	1642	0.168	0.435	1.01
Gaussian 5×5 + Laplacian	26.61	0.501	141.8	0.173	0.377	1.19
Median 3×3 + Laplacian	31.84	0.920	42.6	0.356	0.843	1.12

Table 3: Laplacian sharpening of the blurred image (reference: the clean image).

Image	PSNR (dB)	SSIM	Edge-F1	Sharpness (var ∇^2)
Original	–	–	–	253.4
Blurred (Gaussian 5×5, σ=1.5)	33.55	0.921	0.487	7.3
Sharpened $\alpha = 0.5$	34.36	**0.930**	0.634	33.4
Sharpened $\alpha = 1.0$	**34.62**	0.925	0.739	82.9
Sharpened $\alpha = 2.0$	33.25	0.881	**0.853**	259.9

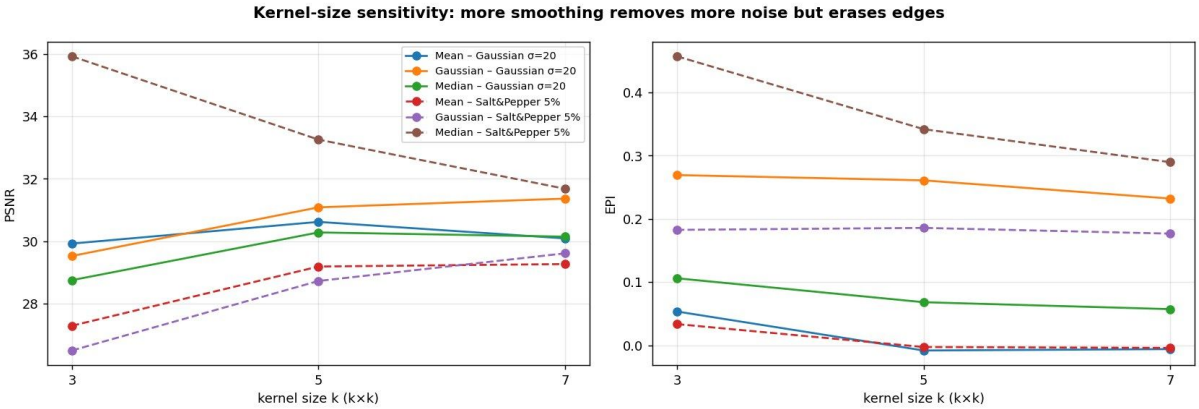


Figure 7: Kernel size sensitivity ($k = 3, 5, 7$): PSNR and Edge-F1 for each filter family.

Table 4: Summary of strengths and weaknesses.

Filter	Strengths	Weaknesses
Mean	Simplest and fastest; fine for mild Gaussian noise with $k = 3$	Blurs edges quickly (Edge-F1 0.07 at 7×7); useless for impulses
Gaussian	Best for Gaussian noise (31.4 dB); no ringing; σ gives fine control	Still blurs edges; spreads impulses into blobs
Median	Best for salt-and-pepper (35.9 dB); keeps step edges	Weaker for Gaussian noise; slow for $k \geq 7$; removes thin lines
Laplacian	Restores edges of a blurred image (Edge-F1 0.49→0.74)	Amplifies noise; halos when α is large; must follow denoising

3 Part B: 3D Reconstruction

3.1 Methodology

Epipolar geometry and the fundamental matrix. When the same 3D point is seen at $\mathbf{x}$ in the left image and at $\mathbf{x}'$ in the right image (homogeneous coordinates), both views satisfy [4]

$$\mathbf{x}'^{\top} F\,\mathbf{x} = 0, \qquad \operatorname{rank}(F) = 2, \tag{5}$$

where F is the 3×3 fundamental matrix. $\mathbf{l}' = F\mathbf{x}$ is the epipolar line on which $\mathbf{x}'$ must lie. Each match gives one linear equation in the 9 entries of F, so 8 matches are enough (*8-point algorithm* with Hartley normalisation). Real matches contain outliers, so F is estimated with *RANSAC* [8]. RANSAC repeatedly fits F to a random minimal sample, counts the matches whose distance to the epipolar line is below a threshold (1 px), and keeps the model with the most inliers. *LMedS* instead minimises the median of the squared residuals. Its accuracy is measured with the distance to the epipolar line, and with the Sampson error (a first-order approximation of the geometric error):

$$e_{\mathrm{S}} = \frac{|\mathbf{x}'^{\top} F\mathbf{x}|}{\sqrt{(F\mathbf{x})_1^2 + (F\mathbf{x})_2^2 + (F^{\top}\mathbf{x}')_1^2 + (F^{\top}\mathbf{x}')_2^2}}. \tag{6}$$

Matches come from SIFT features with Lowe's ratio test. For a rectified pair, every epipolar line must be horizontal, so the mean tilt of the lines is also reported.

Disparity and depth. In a rectified pair, corresponding points are on the same row, and their horizontal shift $d = x_L - x_R$ is the disparity. By similar triangles, $Z = fB/d$ (f focal length in pixels, B baseline). The Middlebury frames are crops whose x-offset differs by d_{offs} pixels (10 px for Venus and 26 px for Teddy), so

$$Z = \frac{f\,B}{d + d_{\mathrm{offs}}}, \qquad X = \frac{(u - c_x)Z}{f}, \qquad Y = \frac{(v - c_y)Z}{f}. \tag{7}$$

In OpenCV this is `cv2.reprojectImageTo3D` with

$$Q = \begin{bmatrix} 1 & 0 & 0 & -c_x \\ 0 & 1 & 0 & -c_y \\ 0 & 0 & 0 & f \\ 0 & 0 & 1/B & d_{\mathrm{offs}}/B \end{bmatrix}, \qquad [X\ Y\ Z\ W]^{\top} = Q\,[u\ v\ d\ 1]^{\top}. \tag{8}$$

Because of the offset, the measured disparity can be negative. The search range $[d_{\min}, d_{\min} + \text{numDisparities})$ is therefore estimated automatically from the horizontal shift of the RANSAC inlier matches (1st to 99th percentile, ± 4 px margin, rounded up to a multiple of 16). This gives $[-11, 21)$ for Venus and $[-15, 33)$ for Teddy.

Block Matching (BM). For each pixel, a $b \times b$ block is compared with blocks on the same row of the right image using the sum of absolute differences (SAD), and the disparity with the lowest cost wins (winner-takes-all). The method is local: it is fast, but it fails where the block has no texture or crosses a depth edge. Invalid results are removed with a texture threshold, a uniqueness ratio of 10% and a speckle filter.

Semi-Global Matching (SGBM) [9] adds a smoothness term and minimises

$$E(D) = \sum_{p} C(p, D_p) + \sum_{q \in N_p} P_1\,[\,|D_p - D_q| = 1\,] + \sum_{q \in N_p} P_2\,[\,|D_p - D_q| > 1\,], \tag{9}$$

where C is the matching cost, P_1 penalises small disparity changes (slanted surfaces), and P_2 penalises jumps (depth edges). The energy is minimised by dynamic programming along 8 directions, and the costs are summed. Following OpenCV's recommendation, $P_1 = 8 \cdot 3 \cdot b^2$ and $P_2 = 32 \cdot 3 \cdot b^2$, in mode `SGBM_3WAY`.

Metrics.

- *Valid disparity ratio*: the percentage of pixels with a disparity.
- *LR consistency*: the percentage of valid pixels whose left and right disparities agree within 1 px.
- *Bad-δ* [10]: the percentage of ground-truth pixels with $|d - d_{gt}| > \delta$, where a missing pixel counts as bad (the standard Middlebury rule). We also report Bad-1 computed only on the estimated pixels.
- *EPE* (mean $|d - d_{gt}|$) and *RMSE* on the estimated pixels.
- For F: the number of inliers, the mean epipolar distance, the mean Sampson error, the line tilt, and the runtime.

3.2 Implementation and Results

Data. As required, the stereo images come from the Middlebury optical-flow page [13, 12]. Two of its scenes are real stereo pairs: **Venus** (`other-color-twoframes.zip`, 420×380) and **Teddy** (`eval-color-twoframes.zip`, 420×360). In both, `frame10` is the left view and `frame11` the right view. The vertical flow is 0, and the horizontal flow is $u = -d$. The Venus ground truth is therefore read from `other-gt-flow/Venus/flow10.flo` with $d = -u$. The Teddy flow ground truth is hidden (it is an evaluation scene), so the public Middlebury stereo ground truth `disp2.png` [11] is cropped to the same window and used for evaluation only. No calibration is given, so $f = 0.8\,W = 336$ px, $c_x = W/2$, $c_y = H/2$ and $B = 1$ are used. The 3D shape is correct, but the scale is relative.

Implementation. The pipeline uses `cv2.SIFT_create` and `cv2.BFMatcher` with a ratio test, then `cv2.findFundamentalMat` (FM_8POINT, FM_RANSAC with 1 px and 0.999 confidence, FM_LMEDS) and `cv2.computeCorrespondEpilines` for drawing. Disparity is computed with `cv2.StereoBM_create` (gray) and `cv2.StereoSGBM_create` (colour). OpenCV returns fixed-point values $\times 16$, so they are divided by 16, and the invalid pixels ($< d_{\min}$) are set to NaN. For the LR check, the right disparity map is computed by mirroring both images. The point cloud is saved as a binary `.ply` file (it opens in MeshLab or Open3D and can be rotated) and as an interactive Plotly `.html` file (rotate with the mouse in any browser). Static screenshots from three viewpoints are included below.

The RANSAC estimate for Venus (normalised so that $F_{33} = 1$) is

$$F = \begin{bmatrix} 8.91 \times 10^{-7} & 3.56 \times 10^{-5} & -7.64 \times 10^{-2} \\ -2.69 \times 10^{-5} & 7.24 \times 10^{-6} & -6.17 \times 10^{-1} \\ 7.42 \times 10^{-2} & 6.10 \times 10^{-1} & 1 \end{bmatrix}.$$

Its third singular value is about 10^{-17} of the first, so it is rank 2, as expected. The large entries $F_{23} \approx -F_{32}$ correspond to the form of F for a pure horizontal translation, which confirms that the pair is almost rectified.

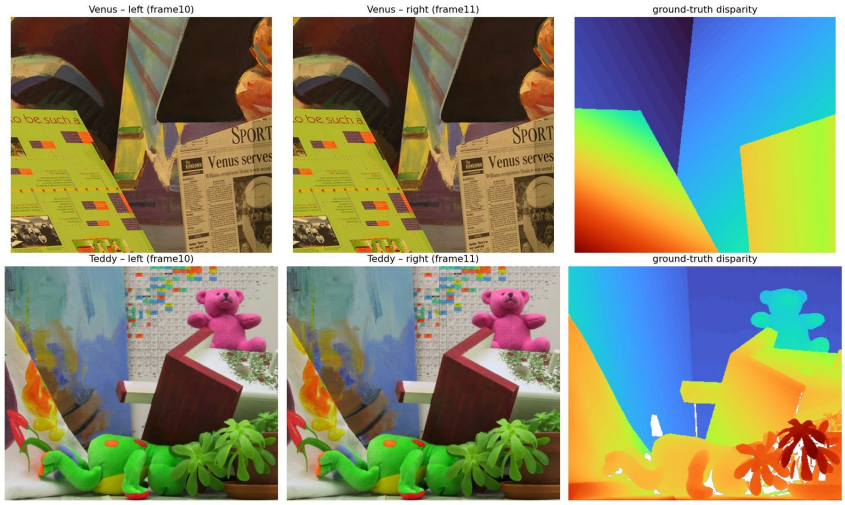


Figure 8: Input pairs from the Middlebury flow dataset (left = `frame10`, right = `frame11`).

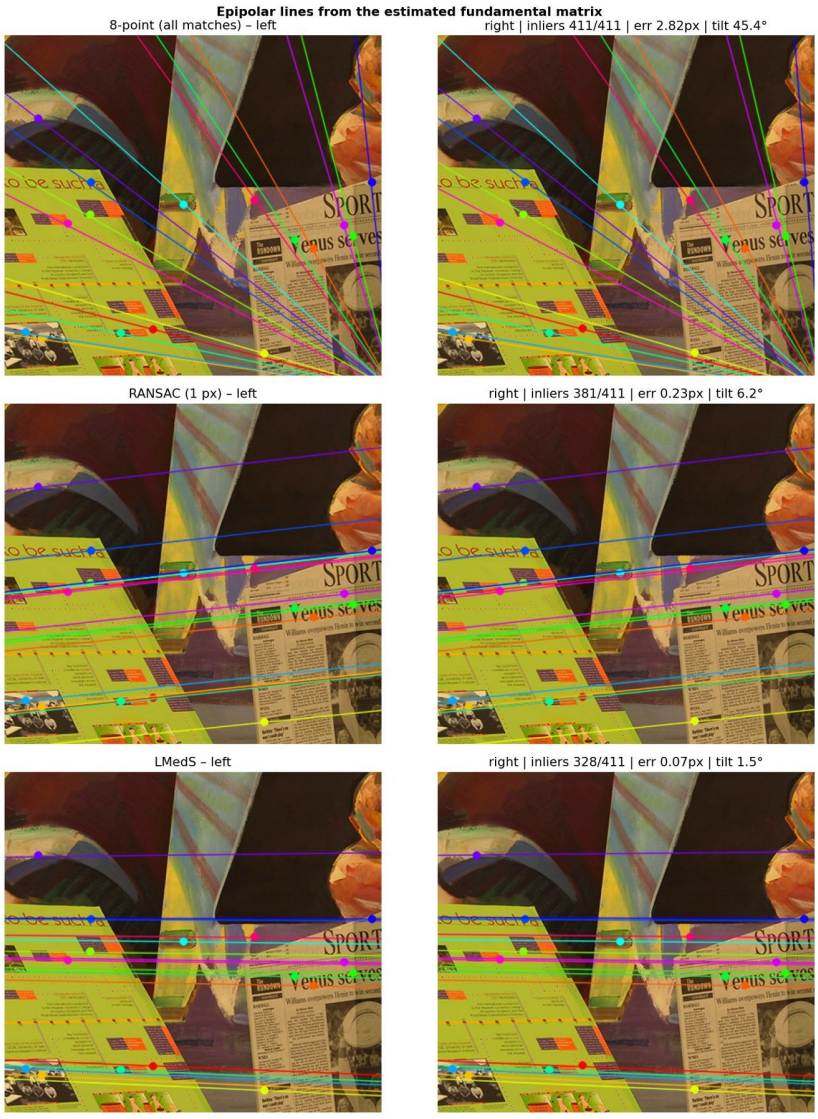
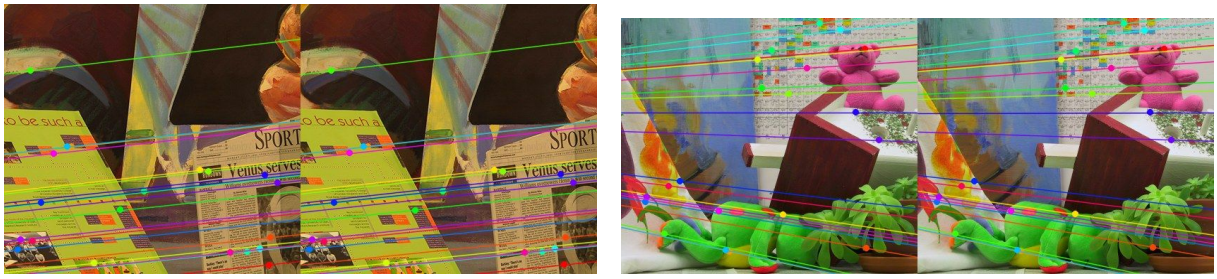


Figure 9: Venus: epipolar lines from F estimated by the 8-point algorithm (top), RANSAC (middle) and LMedS (bottom). The same points are drawn in all rows. Each point must lie on its line in the other image.



(a) Venus (b) Teddy

Figure 10: RANSAC fundamental matrix: 20 inlier matches with their epipolar lines on the left and right images.

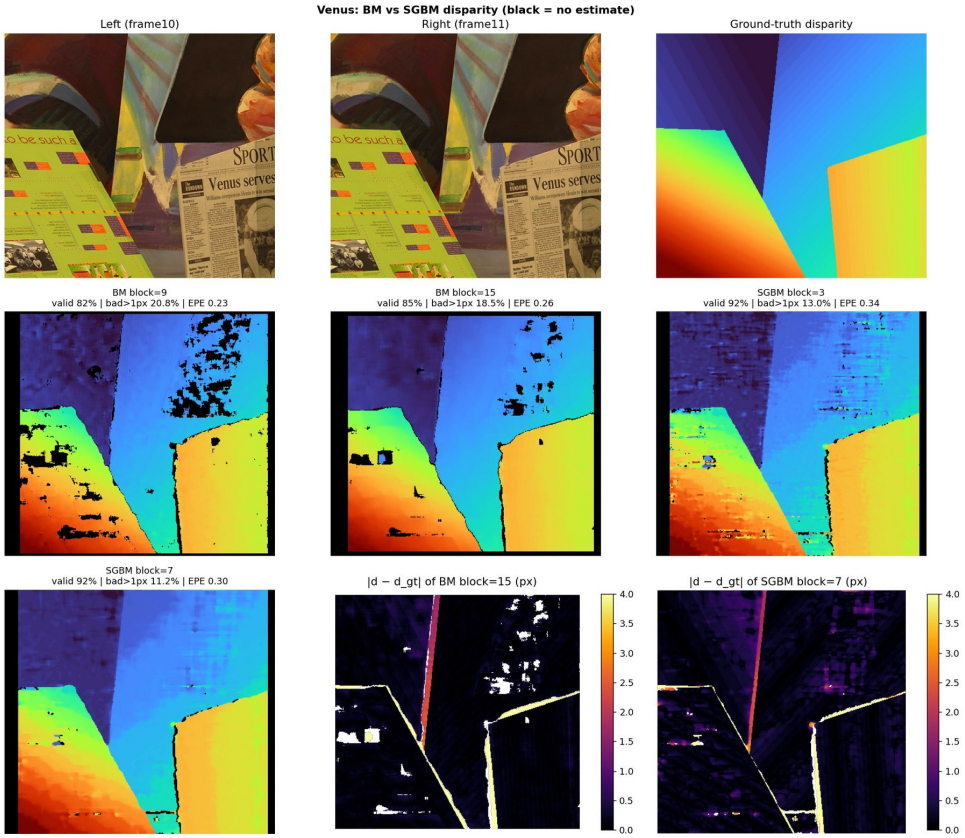


Figure 11: Venus: ground truth, BM (blockSize 9, 15) and SGBM (blockSize 3, 7). Black means no estimate. The last two panels show the absolute error.

Table 5: BM against SGBM on both scenes (search range estimated automatically).

Scene	Method	Valid %	LR %	Bad-1 %	Bad-2 %	Bad-1 est. %	EPE (px)	Time (ms)
Venus	BM, block 9	81.6	93.4	20.8	20.3	**2.9**	**0.23**	6.97
	BM, block 15	84.6	94.3	18.5	17.9	3.7	0.26	**6.41**
	SGBM, block 3	91.5	95.4	13.1	10.6	5.0	0.34	10.99
	SGBM, block 7	**91.7**	**96.3**	**11.2**	**10.3**	3.2	0.30	12.30
Teddy	BM, block 9	78.9	92.0	27.2	26.0	**7.9**	0.67	8.55
	BM, block 15	78.6	91.1	28.8	27.5	9.4	0.75	**8.13**
	SGBM, block 3	**86.1**	93.7	**21.1**	**18.8**	8.1	**0.59**	15.78
	SGBM, block 7	86.0	**93.8**	21.9	19.6	8.9	0.64	17.26

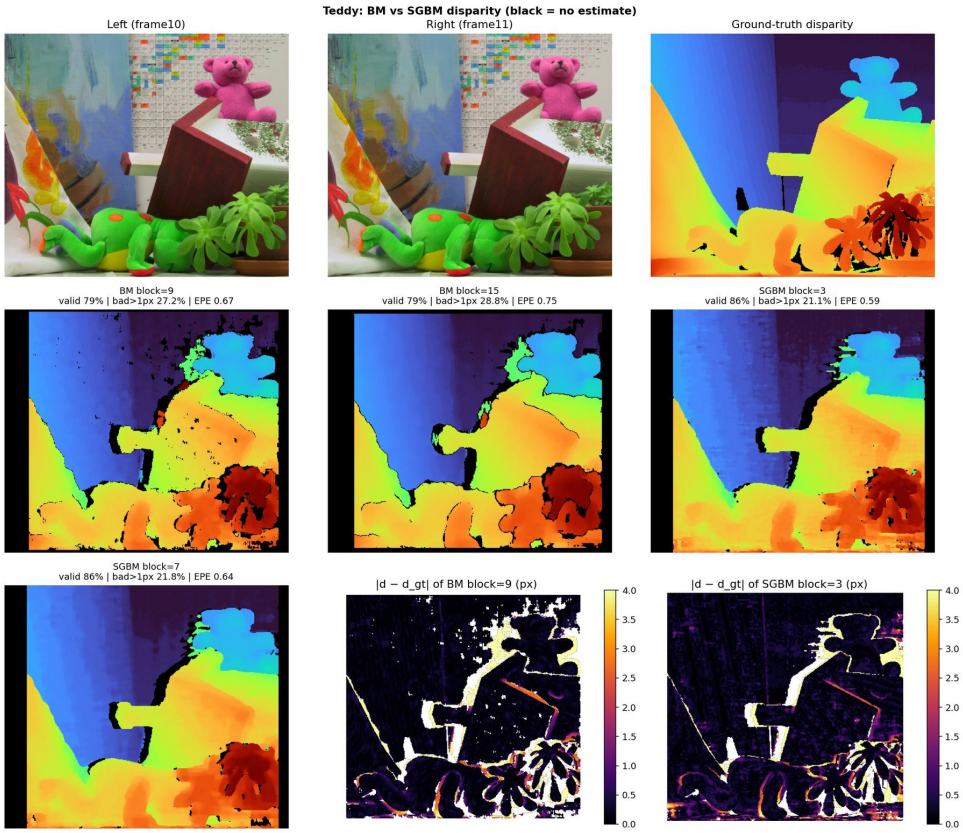
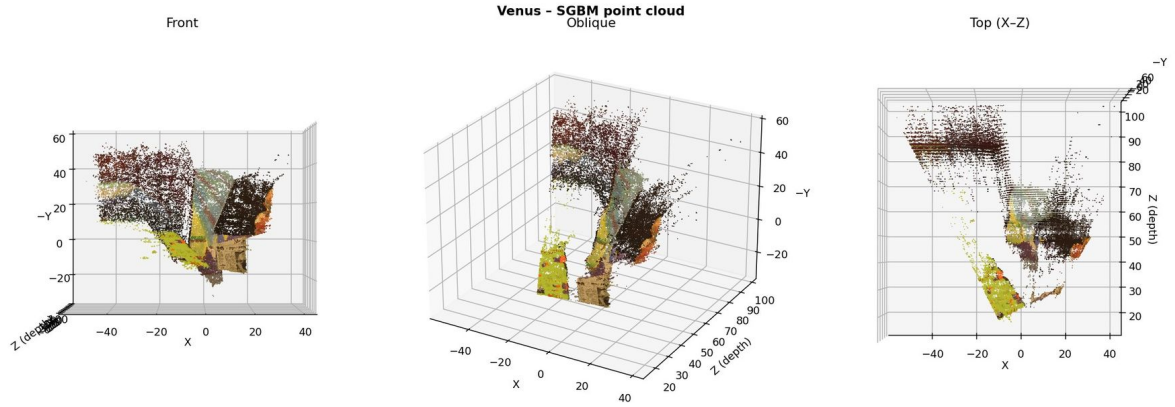


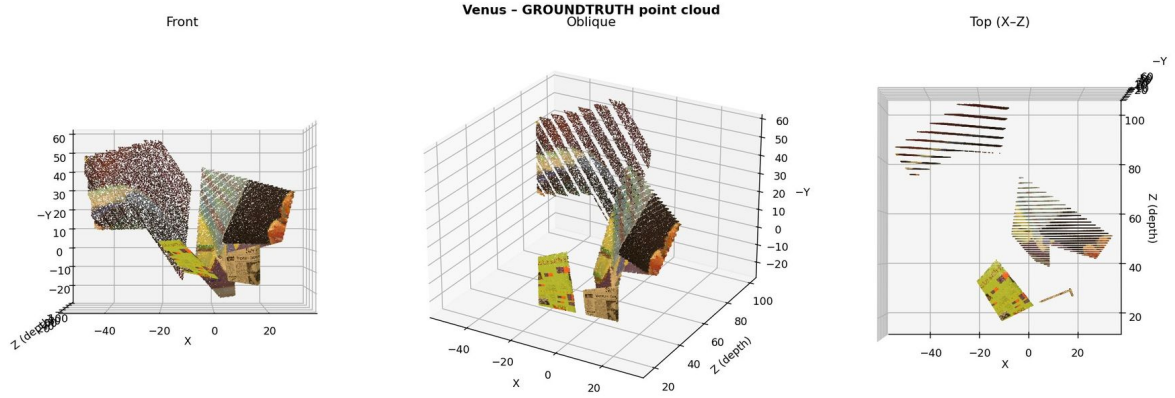
Figure 12: Teddy: the same comparison on a scene with more depth edges and occlusions.

Table 6: Estimating the fundamental matrix from the same SIFT matches (Venus: 411, Teddy: 353).

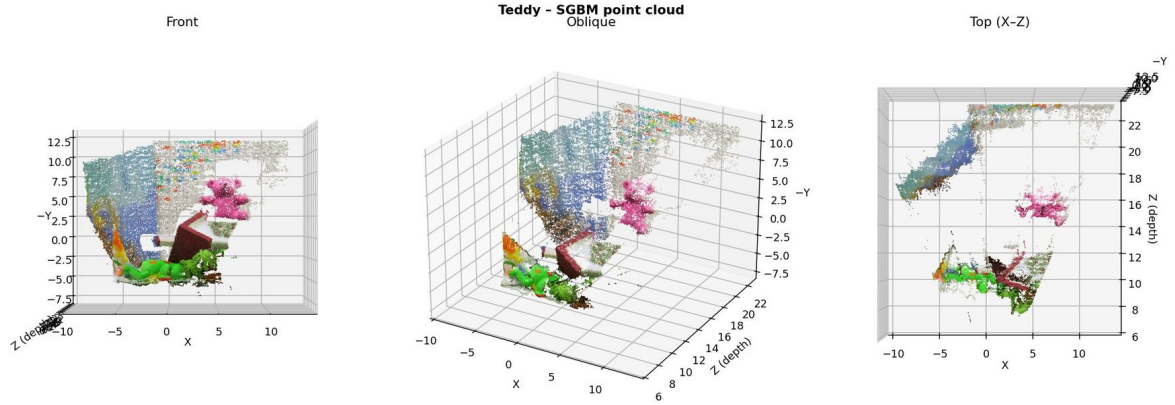
Scene	Estimator	Inliers	Epi. err (px)	Sampson (px)	Line tilt	Time (ms)
Venus	8-point (all)	411 (100%)	2.825	1.810	45.4°	0.21
	RANSAC 1 px	381 (92.7%)	0.233	0.165	6.2°	0.81
	LMedS	328 (79.8%)	0.066	0.047	1.5°	27.64
Teddy	8-point (all)	353 (100%)	2.202	1.538	36.1°	0.22
	RANSAC 1 px	286 (81.0%)	0.262	0.185	5.2°	2.52
	LMedS	285 (80.7%)	0.113	0.080	1.0°	27.76



(a) Venus – SGBM point cloud



(b) Venus – point cloud from the ground-truth disparity



(c) Teddy – SGBM point cloud

Figure 13: Coloured 3D point clouds (front, oblique and top views). The rotatable versions are `B2_point_cloud_*_interactive.html` and `point_cloud_*.ply` in the code package.

Table 7: Parameter study (Venus range $[-11, 21)$, Teddy range $[-15, 33)$). blockSize is varied at the default range; numDisparities is varied with BM block 9 and SGBM block 5.

		Venus		Teddy	
Algorithm	Setting	Valid %	Bad-1 %	Valid %	Bad-1 %
BM	blockSize 5	71.1	31.0	71.8	32.9
BM	blockSize 9	81.6	20.8	78.9	**27.2**
BM	blockSize 15	84.6	**18.5**	78.6	28.8
BM	blockSize 21	84.0	19.7	77.0	31.1
SGBM	blockSize 3	91.5	13.1	86.1	**21.1**
SGBM	blockSize 5	91.5	11.7	86.1	21.3
SGBM	blockSize 7	91.7	**11.2**	86.0	21.9
SGBM	blockSize 11	91.6	12.0	85.6	24.2
BM	numDisparities 16	82.2	21.4	56.0	58.9
BM	numDisparities 32	81.6	**20.8**	83.2	**23.7**
BM	numDisparities 48	77.2	25.0	78.9	27.2
BM	numDisparities 64	73.2	28.9	74.7	31.2
SGBM	numDisparities 16	94.1	14.7	82.8	57.0
SGBM	numDisparities 32	91.5	**11.7**	89.9	**18.0**
SGBM	numDisparities 48	87.7	15.5	86.1	21.3
SGBM	numDisparities 64	83.9	19.1	82.2	25.1

3.3 Comparative Analysis

BM against SGBM.

- *Density and overall accuracy.* SGBM gives a disparity for 91.7% of the Venus pixels, against 84.6% for BM, and lowers the Bad-1 rate from 18.5% to 11.2%. On Teddy, Bad-1 drops from 27.2% to 21.1%. BM leaves holes wherever a block has no texture (the dark top-right of the painting in Venus, and the flat walls in Teddy), because SAD has no clear minimum there. SGBM fills these regions from the neighbouring pixels through the smoothness term.
- *Accuracy on the estimated pixels.* Where BM does return a value, it is very accurate (Bad-1 2.9% on Venus, against 3.2–5.0% for SGBM). The texture and uniqueness checks of BM simply reject the hard pixels, while SGBM fills them with smoothed guesses, which are sometimes slightly wrong. This is why both "Bad-1" columns are reported.
- *Smoothness and artifacts.* The SGBM maps are piecewise smooth, with visible horizontal streaks (Figure 11) caused by the directional aggregation. BM maps have holes and blocky edges. Both methods "fatten" foreground objects at depth edges, which appears as bright lines in the error maps. In the 3D clouds this produces flying points between the foreground and background planes. Because $Z \propto 1/d$, the far background is split into separate depth layers (top view in Figure 13), while the cloud built from the ground truth (sub-pixel accurate) shows clean planes.
- *Runtime.* BM takes about 7 ms and SGBM 11–17 ms, so SGBM is roughly 1.6 to 2 times slower for a large gain in density and Bad-1. SGBM also has the best LR consistency (96.3% on Venus).

Effect of blockSize. BM is very sensitive to it. A 5×5 block is too small to be distinctive (Bad-1 31%), while 21×21 starts to fatten edges again (19.7%), so the best value is 9–15. SGBM is almost insensitive to it (11.2–13.1% on Venus), because the smoothness term already regularises the result. Only a large 11×11 block hurts on Teddy (24.2%), where depth edges are frequent.

Effect of numDisparities. For each extra 16 disparities, the matcher cannot evaluate a band of 16 more pixels at the left image border, so the valid ratio drops by about 4% and Bad-1 increases (Venus SGBM: 11.7% at 32, 19.1% at 64). If the range is too small, the true disparity is cut off. On Teddy, 16 disparities give Bad-1 of 57–59%. On Venus, SGBM with 16 disparities looks dense (94%), but its error is high (EPE 0.68 px and Bad-1 on the estimated pixels 9.4%, against 0.30 px and 3.2% at 32), because it returns confident but wrong values. The range should therefore cover the scene and be no larger. This is why it is estimated from the SIFT matches rather than set by hand. On Teddy, 32 is slightly better than the estimated 48, because only a few pixels lie beyond $d = 16$, and that loss is smaller than the 16-pixel border band.

Fundamental matrix. The 8-point algorithm fitted to all matches is pulled by the outliers. Its lines are tilted by 36–45°, although the pair is almost rectified, and the mean error is 2.2–2.8 px. RANSAC rejects 7–19% of the matches and reduces the error by a factor of about 10 (0.23 px, Sampson 0.16 px) at under 3 ms. LMedS fits the inliers even more tightly (0.07 px, tilt 1.5°), but it is about 35 times slower and only works if at least half of the matches are inliers. The remaining tilt of RANSAC (5–6°) comes from the scene structure: Venus consists of a few planes, and points on one plane give a nearly degenerate configuration for F. In practice, F from RANSAC is accurate enough to verify matches and to rectify, but a fixed stereo rig should be calibrated instead.

4 Part C: Image Stitching

4.1 Methodology

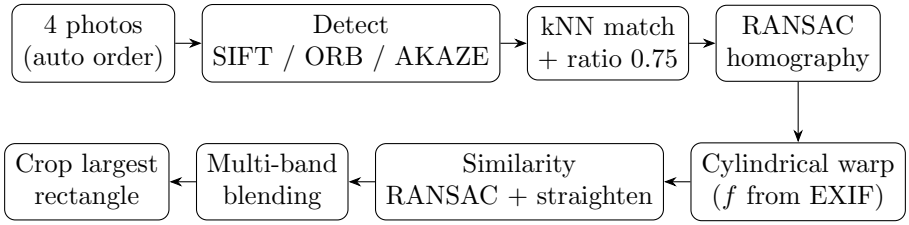


Figure 14: Stitching pipeline used in Part C.

Feature detection and description.

- **SIFT** [14] finds extrema of the Difference-of-Gaussians scale space, and describes each one with a 128-D histogram of gradient orientations (floating point, L_2 distance). It is invariant to scale and rotation, and is accurate but slow.
- **ORB** [15] detects FAST corners on an image pyramid, assigns them an orientation from the intensity centroid, and describes them with a rotated 256-bit BRIEF descriptor (Hamming distance). It is very fast and cheap to match.
- **AKAZE** [16] builds a non-linear diffusion scale space, which smooths flat regions but keeps edges, and uses a binary M-LDB descriptor (Hamming distance).

Matching. A brute-force matcher returns the two nearest neighbours m_1, m_2 of every descriptor (KNN, $k = 2$). A match is kept only if it passes Lowe's ratio test, $\|d - d_{m_1}\| < 0.75\,\|d - d_{m_2}\|$. This rejects ambiguous matches, for example on the repeated windows of a building.

Homography. Two views of a distant scene, or two views taken by a camera rotating about its centre, are related by a projective transformation with 8 degrees of freedom [4]:

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} \sim H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \qquad H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}. \tag{10}$$

Each match gives 2 equations, so 4 matches determine H (DLT). RANSAC samples 4 matches, fits H, and counts as inliers the matches with reprojection error $\|\mathbf{x}' - H\mathbf{x}\| < t$. After $N = \log(1-p)/\log(1-w^4)$ iterations (w the inlier ratio, $p = 0.999$), the best model is refitted on all of its inliers. The thresholds $t = 1, 3, 5, 10$ px are compared, and $t = 3$ px is used in the main pipeline.

Warping and projection. A planar mosaic warps every image into the frame of the middle image with `cv2.warpPerspective`. For a wide field of view, this stretches the outer images strongly (a "bow-tie" shape). The *cylindrical* projection first maps every image onto a cylinder of radius f:

$$\theta = \frac{x - c_x}{f}, \qquad h = \frac{y - c_y}{\sqrt{(x - c_x)^2 + f^2}}\, f, \qquad (x_c, y_c) = (f\theta + c_x,\ h + c_y). \tag{11}$$

On the cylinder, a rotation of the camera about the vertical axis becomes a horizontal translation, so the images are aligned with a 4-DoF similarity transform (RANSAC), and the small remaining tilt is removed ("straightening"). The focal length in pixels comes from the EXIF data: $f_{px} = \frac{f_{35}}{36} W = \frac{26}{36} \cdot 1200 = 866.7$ px for the iPhone main camera. The focal length that can be derived from the homographies (Szeliski–Shum [17]) varies between 722 and 878 px depending on the detector, so it is only reported for comparison.

Blending. Three methods are compared.

- *Overwrite*: the last image is pasted on top.
- *Feathering*: in the overlap, each image is weighted by its distance to its own border (`cv2.distanceTransform`).
- *Multi-band blending* [19, 18]: each image is split into a 5-level Laplacian pyramid. Each band is blended with a Gaussian-pyramid version of the seam mask, and the result is rebuilt. Low frequencies are mixed over a wide zone (which hides exposure differences), and high frequencies over a narrow zone (which avoids ghosting).

Finally, the largest axis-aligned rectangle without black pixels is cropped.

Metrics. The metrics are the keypoints per image, the number of initial (ratio-test) matches, the RANSAC inliers and the inlier ratio, the reprojection RMSE of the inliers, $\sqrt{\frac{1}{n}\sum \|\mathbf{x}'_i - H\mathbf{x}_i\|^2}$, and the total stitching time (detection to final crop). For the panorama, we also report the percentage of black border before the crop, a *straightness* score (the vertical spread of the top and bottom edges relative to the height; 0 means a perfectly flat strip), and the area kept after the crop.

4.2 Implementation and Results

Photos. The four photos (`IMG_3111`–`IMG_3114`, Figure 15) were taken with my phone (iPhone 12, 26 mm main camera) from the window at home. I turned the phone around one fixed point and kept a large overlap between neighbouring photos. The originals are 3024×1701 (HEIC converted to JPEG with the EXIF data kept) and are resized to 1200×675 for speed. The file names are not in left-to-right order. The program recovers the order automatically, by placing each image with a least-squares fit of the pairwise translations (result: 3112, 3111, 3113, 3114).

Implementation. The pipeline uses `cv2.SIFT_create`, `cv2.ORB_create` (6000 features), `cv2.AKAZE_create`, `cv2.BFMatcher` (NORM_L2 for SIFT, NORM_HAMMING for the binary descriptors; exact search, so the results are deterministic), `knnMatch(k=2)` with the ratio test, `cv2.findHomography(RANSAC)`, `cv2.remap` for the cylindrical projection, `cv2.estimateAffinePartial2D` for the similarity, `cv2.warpPerspective`, `cv2.pyrDown` and `cv2.pyrUp` for the pyramids, and `cv2.distanceTransform`.

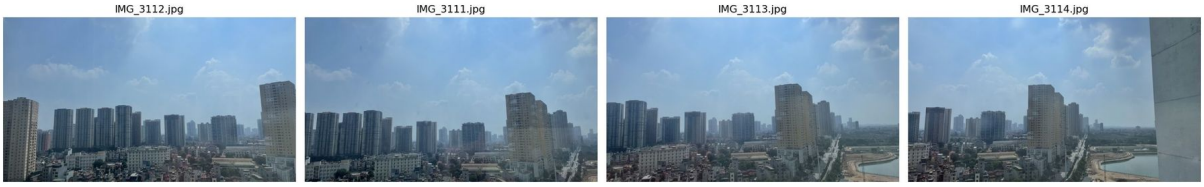


Figure 15: The four overlapping photos taken from my home, in the recovered left-to-right order.

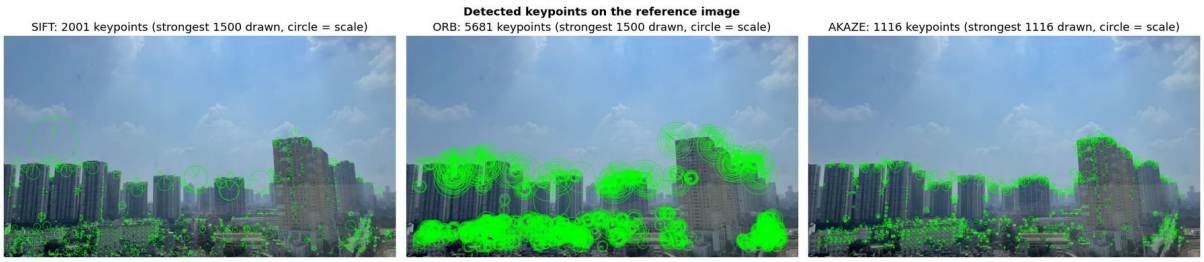


Figure 16: Keypoints detected on the reference (middle) image by SIFT, ORB and AKAZE (circle size = scale). ORB keypoints are concentrated in clusters on the most textured areas (rooftops), while SIFT and AKAZE are spread more evenly over the buildings. No detector fires on the sky.

4.3 Comparative Analysis

Table 8: Detector comparison over the 3 image pairs (RANSAC threshold 3 px; time = whole cylindrical stitching pipeline).

Detector	Keypts/img	Init. matches	Inliers	Inlier %	RMSE (px)	Time (ms)
SIFT	2241	2111	1925	91.2	**0.97**	2362
ORB	**5802**	**2918**	**2583**	88.5	1.40	1432
AKAZE	1359	1390	1319	**94.9**	0.99	**919**

SIFT, ORB and AKAZE.

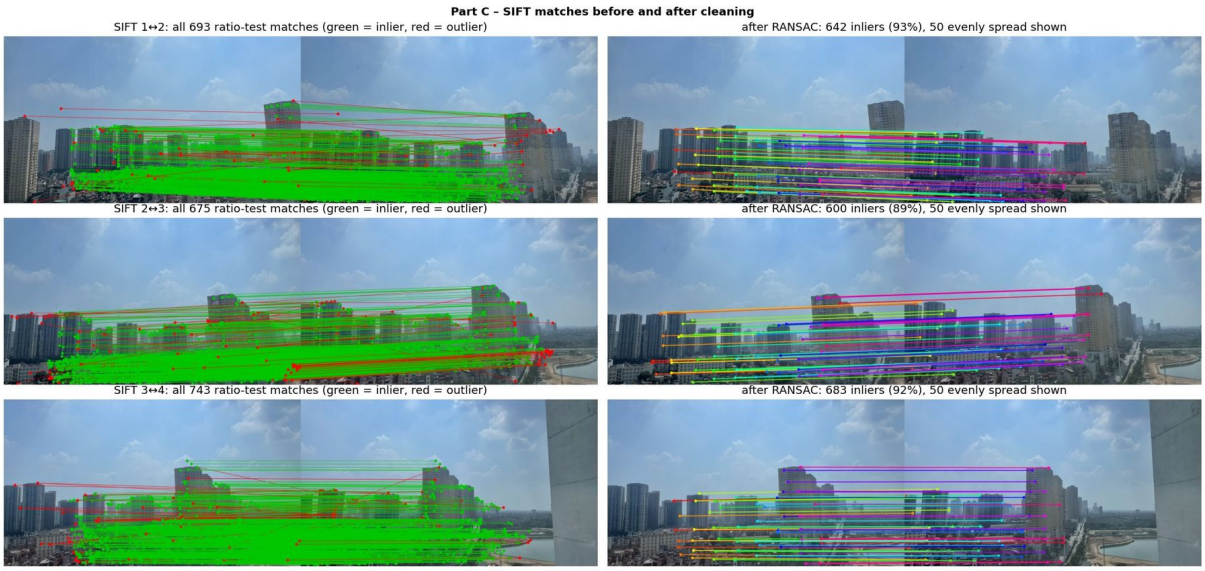


Figure 17: SIFT matches for the three pairs. Left: all ratio-test matches (green = RANSAC inlier, red = outlier). Right: a spread-out subset of the inliers.

Table 9: RANSAC threshold study: inlier ratio (%) / reprojection RMSE (px).

Detector	1 px	3 px	5 px	10 px
SIFT	67.7 / 0.54	91.2 / 0.97	93.8 / 1.13	94.5 / 1.23
ORB	42.1 / 0.63	88.5 / 1.40	96.1 / 1.57	97.5 / 1.69
AKAZE	65.1 / 0.52	94.9 / 0.99	96.6 / 1.12	97.7 / 1.30

Table 10: Homography estimator on SIFT pair 1–2: RMSE (px) on the true inliers, with the ratio-test matches and after adding 20% random outliers.

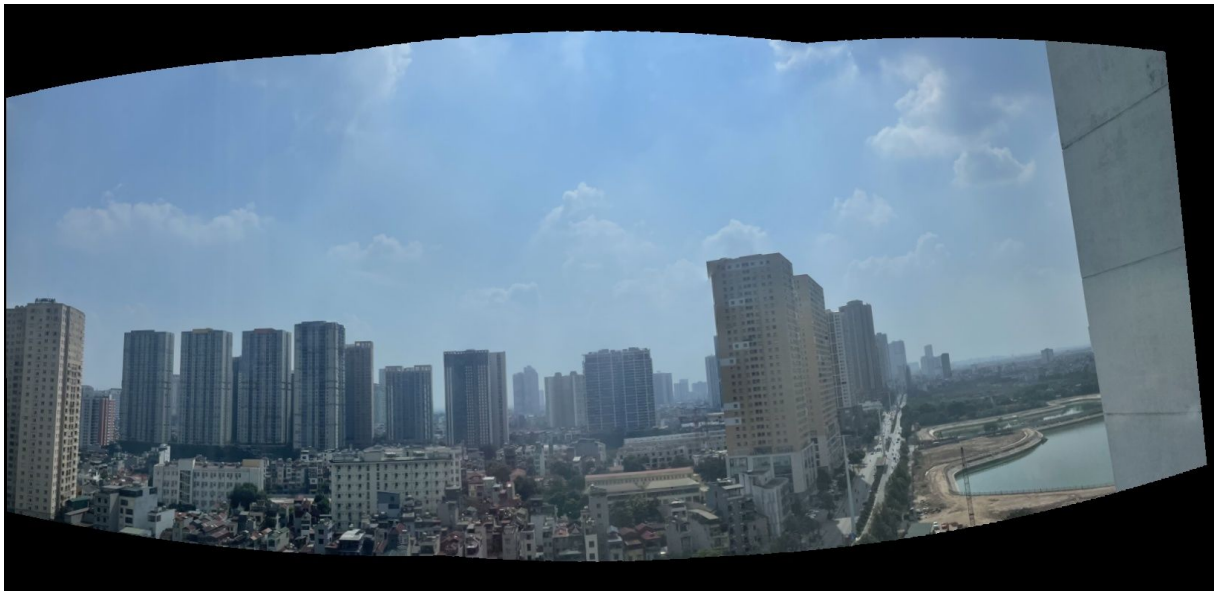
Estimator	Ratio-test matches (693)	+20% outliers (831)
Least squares (all matches)	18.62	195.3
RANSAC, t=1 px	0.46 (463 inliers)	0.48
RANSAC, t=3 px	0.85 (642 inliers)	0.63
RANSAC, t=10 px	0.84 (654 inliers)	0.85
LMedS	0.50 (608 inliers)	0.60

Table 11: Planar against cylindrical projection (cropped size, and the time for the whole pipeline).

Detector	Projection	Black border %	Straightness %	Area kept %	Final size	Time (ms)
SIFT	planar	25.0	29.0	85.7	2170 × 705	1351
SIFT	cylindrical	18.3	11.9	76.2	1563 × 512	2362
ORB	planar	23.3	16.9	85.8	2225 × 705	1442
ORB	cylindrical	18.5	11.7	76.6	1560 × 515	1432
AKAZE	planar	25.1	26.7	85.6	2193 × 706	1498
AKAZE	cylindrical	18.2	11.7	76.5	1554 × 515	919



(a) Planar projection (homographies to the middle image), before cropping



(b) Cylindrical projection ($f = 866.7$ px) + straightening, before cropping

Figure 18: Planar against cylindrical projection (SIFT). The planar mosaic stretches the outer images into a bow-tie, and the wall on the right is strongly enlarged.

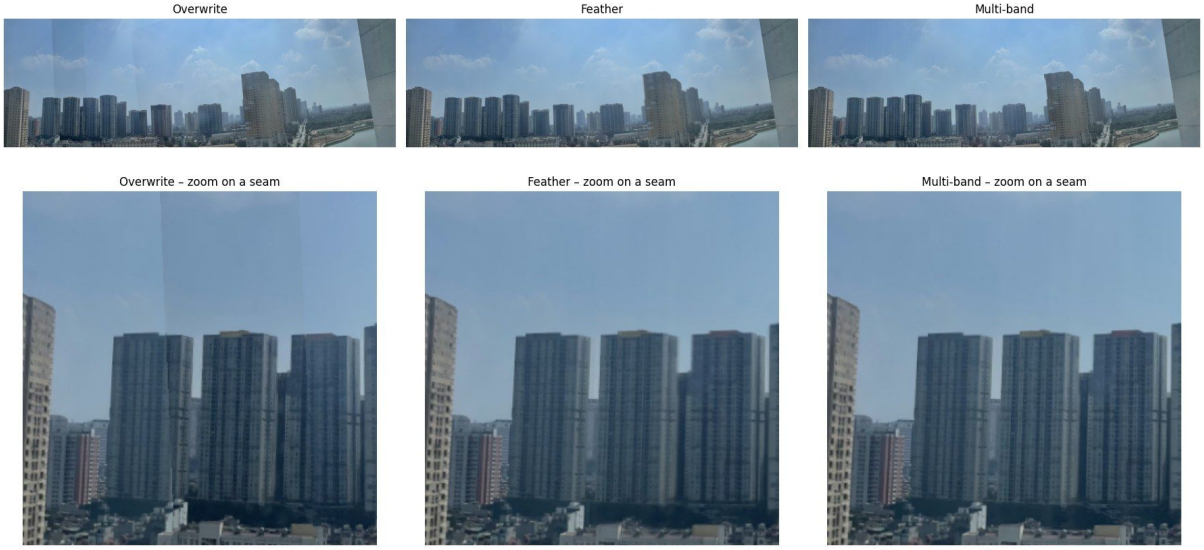


Figure 19: Blending comparison (SIFT + cylindrical). Top: full panoramas; bottom: zoom on a seam.



Figure 20: Final panorama for each detector (cylindrical, multi-band blending, cropped).

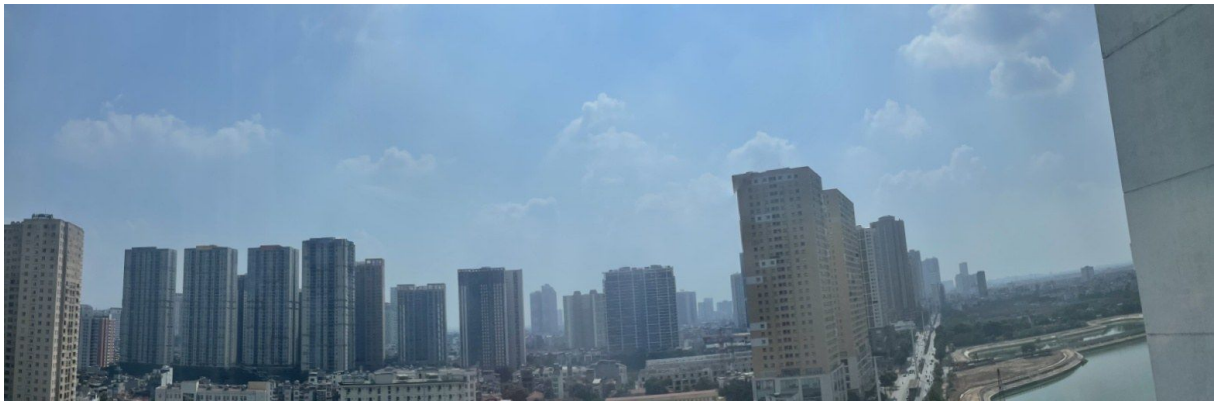


Figure 21: Final panorama (SIFT + cylindrical projection + multi-band blending, 1563 × 512).

- **ORB** finds the most keypoints (5802 per image, since the limit is 6000) and the most matches, but has the lowest inlier ratio (88.5%) and the largest RMSE (1.40 px). Its binary descriptor is less distinctive, and FAST corners on a coarse pyramid (scale factor 1.2) are located less precisely. It still produces a correct panorama, because thousands of inliers remain.
- **SIFT** is the most precise (0.97 px) thanks to sub-pixel keypoint refinement and its 128-D float descriptor. It is the slowest (2.4 s for the whole pipeline, which detects features twice: once for the homographies and once on the cylinder images).
- **AKAZE** has the fewest keypoints (1359) but the cleanest matches (94.9% inliers, 0.99 px) and the shortest total time (0.92 s). Its edge-preserving scale space gives stable features on the building facades.

All three detectors give visually the same panorama (Figure 20). For this kind of scene, AKAZE is the best trade-off between speed and accuracy, and SIFT is the safest choice when accuracy matters most.

RANSAC threshold. At 1 px, RANSAC is too strict: correct matches with normal localisation noise are rejected (only 42% for ORB). Raising the threshold to 3 px brings the inlier ratio to 88–95% with sub-pixel to 1.4 px RMSE. Beyond that, few new inliers are added, while the RMSE keeps increasing (ORB 1.40→1.69 px), which means that slightly wrong matches are being accepted. So 3 px is the sweet spot. Table 10 shows why a robust estimator is needed at all: a least-squares fit to all ratio-test matches already has 18.6 px error, and 195 px when 20% outliers are added, while RANSAC and LMedS stay below 1 px in both cases.

Planar against cylindrical projection. The planar mosaic keeps more area (86%), but it bends into a bow-tie (straightness 29% with SIFT), and the concrete wall on the right is stretched a lot, because points far from the reference image are projected at large angles. The cylindrical mosaic is much flatter (11.7–11.9% for all detectors) and keeps the vertical lines of the buildings straight. It loses more area when cropped (76%) because I tilted the phone slightly during the capture, which makes the strip wave a little. A wider vertical margin when taking the photos would reduce this.

Seam visibility. With overwrite blending, the seams are visible as steps in brightness in the sky, because the phone changed its exposure from one photo to the next (Fig-

ure 19). Feathering removes the step, but over a wide overlap it slightly blurs fine details. Multi-band blending gives an invisible seam in the sky and keeps the window grid sharp, so it is used for the final result. The remaining photometric difference in the overlaps (about 10–11 gray levels, Overlap-RMSE) comes from exposure and parallax rather than misalignment, since the geometric RMSE is below 1 px.

5 Conclusion

The three parts show that classical methods work well when the method matches the structure of the problem. For filtering, the right filter depends on the noise model: the median filter for impulse noise (35.9 dB) and the Gaussian filter for Gaussian noise (31.4 dB). Kernel size controls the trade-off between noise removal and blur, and SSIM alone can hide the loss of edges, which an edge metric (Edge-F1) reveals. Laplacian sharpening recovers edges only after denoising and with a moderate α; otherwise it amplifies noise or creates halos. For stereo, SGBM gives denser and more accurate disparity than BM (Bad-1 11.2% against 18.5% on Venus) for about twice the runtime, and the disparity range must fit the scene. RANSAC is essential for both the fundamental matrix and the homography, since a least-squares fit fails completely in the presence of outliers. For stitching, AKAZE gives the best trade-off between speed and accuracy, and a cylindrical projection with multi-band blending produces a flat, seamless panorama of my neighbourhood.

Several extensions could improve the results: rectifying the Middlebury pairs from the estimated F, adding a weighted-median or WLS filter to fill the SGBM holes, global bundle adjustment of all four images to avoid the accumulated drift of chained transforms, exposure compensation before blending, and a non-local-means or bilateral filter as a stronger baseline for Part A.

6 References

[1] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000. Documentation: https://docs.opencv.org/4.x/.

[2] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2018.

[3] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.

[4] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press, 2004.

[5] Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: from error visibility to structural similarity," *IEEE Trans. Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[6] F. Sattar, L. Floreby, G. Salomonsson and B. Lövström, "Image enhancement based on a nonlinear multiscale method," *IEEE Trans. Image Processing*, vol. 6, no. 6, pp. 888–895, 1997.

[7] J. Canny, "A computational approach to edge detection," *IEEE Trans. PAMI*, vol. 8, no. 6, pp. 679–698, 1986.

[8] M. A. Fischler and R. C. Bolles, "Random sample consensus: a paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[9] H. Hirschmüller, "Stereo processing by semiglobal matching and mutual information," *IEEE Trans. PAMI*, vol. 30, no. 2, pp. 328–341, 2008.

[10] D. Scharstein and R. Szeliski, "A taxonomy and evaluation of dense two-frame stereo correspondence algorithms," *International Journal of Computer Vision*, vol. 47, pp. 7–42, 2002.

[11] D. Scharstein and R. Szeliski, "High-accuracy stereo depth maps using structured light," in *Proc. IEEE CVPR*, 2003, pp. 195–202.

[12] S. Baker, D. Scharstein, J. P. Lewis, S. Roth, M. J. Black and R. Szeliski, "A database and evaluation methodology for optical flow," *International Journal of Computer Vision*, vol. 92, no. 1, pp. 1–31, 2011.

[13] Middlebury Computer Vision, "Optical flow datasets," https://vision.middlebury.edu/flow/data/, accessed Oct. 2026.

[14] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[15] E. Rublee, V. Rabaud, K. Konolige and G. Bradski, "ORB: an efficient alternative to SIFT or SURF," in *Proc. IEEE ICCV*, 2011, pp. 2564–2571.

[16] P. F. Alcantarilla, J. Nuevo and A. Bartoli, "Fast explicit diffusion for accelerated features in nonlinear scale spaces," in *Proc. British Machine Vision Conference (BMVC)*, 2013.

[17] R. Szeliski and H.-Y. Shum, "Creating full view panoramic image mosaics and environment maps," in *Proc. ACM SIGGRAPH*, 1997, pp. 251–258.

[18] M. Brown and D. G. Lowe, "Automatic panoramic image stitching using invariant features," *International Journal of Computer Vision*, vol. 74, no. 1, pp. 59–73, 2007.

[19] P. J. Burt and E. H. Adelson, "A multiresolution spline with application to image mosaics," *ACM Trans. Graphics*, vol. 2, no. 4, pp. 217–236, 1983.

[20] OpenCV 4.x tutorials: "Smoothing Images," "Depth Map from Stereo Images," "Epipolar Geometry," "Feature Matching + Homography," https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html.

Additional textbooks used for background: Gonzalez and Woods [2] (Part A), Szeliski [3] (all parts), Canny [7] (Edge-F1), and the OpenCV tutorials [20].

A Appendix: Python Code

Core functions only; the full code and the executed notebook are in the folder `23071036_TongKhanhLy_Code`.

A.1 Part A: Image Filtering

```
import cv2
import numpy as np

def add_gaussian_noise(img, sigma=20.0, seed=42):
    rng = np.random.default_rng(seed)
    noisy = img.astype(np.float32) + rng.normal(0.0, sigma,
       img.shape)
```

```
    return np.clip(noisy, 0, 255).astype(np.uint8)

def add_salt_pepper(img, amount=0.05, seed=42):
    r = np.random.default_rng(seed).random(img.shape[:2])
    noisy = img.copy()
    noisy[r < amount / 2] = 0           # pepper
    noisy[r > 1 - amount / 2] = 255     # salt
    return noisy

def laplacian_sharpen(img, alpha=1.0):
    f = img.astype(np.float32)    # keep negative values
    lap = cv2.Laplacian(f, cv2.CV_32F, ksize=1)
    return np.clip(f - alpha * lap, 0, 255).astype(np.uint8)

FILTERS = {
    "Mean 5x5": lambda x: cv2.blur(x, (5, 5)),
    "Gaussian 5x5": lambda x: cv2.GaussianBlur(x, (5, 5), 1.1),
    "Median 3x3": lambda x: cv2.medianBlur(x, 3),
    "Median 3x3 + Lap": lambda x: laplacian_sharpen(
        cv2.medianBlur(x, 3), 0.5),
}

def psnr(ref, img):
    mse = np.mean((ref.astype(float) - img.astype(float)) ** 2)
    return 10 * np.log10(255.0 ** 2 / mse)
```

A.2 Part B: 3D Reconstruction

```
def make_sgbm(block, num_disp, min_disp):
    return cv2.StereoSGBM_create(
        minDisparity=min_disp, numDisparities=num_disp,
           blockSize=block,
        P1=8 * 3 * block ** 2, P2=32 * 3 * block ** 2,
           disp12MaxDiff=1,
        uniquenessRatio=10, speckleWindowSize=100,
           speckleRange=32,
        mode=cv2.STEREO_SGBM_MODE_SGBM_3WAY)

def compute_disparity(matcher, left, right):
    # OpenCV returns fixed point x16; invalid = below min
    d = matcher.compute(left, right).astype(np.float32) / 16
    d[d < matcher.getMinDisparity()] = np.nan
    return d

def disparity_to_points(disp, img, focal, baseline, doffs):
    h, w = disp.shape
    # Z = f*B / (d + doffs)
    Q = np.float32([[1, 0, 0, -w / 2], [0, 1, 0, -h / 2],
                    [0, 0, 0, focal],
                    [0, 0, 1 / baseline, doffs / baseline]])
    d = np.nan_to_num(disp, nan=-1e6)
```

```
    pts = cv2.reprojectImageTo3D(d, Q)
    mask = np.isfinite(disp) & (disp + doffs > 1)
    return pts[mask], img[mask]

F, mask = cv2.findFundamentalMat(p1, p2, cv2.FM_RANSAC, 1.0)
lines = cv2.computeCorrespondEpilines(p1.reshape(-1, 1, 2), 1, F)
```

A.3 Part C: Image Stitching

```
def match(d1, d2, method, ratio=0.75):
    norm = cv2.NORM_L2 if method == "SIFT" else cv2.NORM_HAMMING
    knn = cv2.BFMatcher(norm).knnMatch(d1, d2, k=2)
    # Lowe ratio test
    return [p[0] for p in knn if len(p) == 2
            and p[0].distance < ratio * p[1].distance]

def cylindrical_warp(img, f):
    h, w = img.shape[:2]
    W = int(2 * f * np.arctan(w / 2 / f))
    u, v = np.meshgrid(np.arange(W, dtype=np.float32),
                       np.arange(h, dtype=np.float32))
    theta = (u - W / 2) / f
    map_x = np.float32(f * np.tan(theta) + w / 2)
    map_y = np.float32((v - h / 2) / np.cos(theta) + h / 2)
    return cv2.remap(img, map_x, map_y, cv2.INTER_LINEAR)

sift = cv2.SIFT_create()
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)
good = match(des2, des1, "SIFT")
src = np.float32([kp2[m.queryIdx].pt for m in good])
dst = np.float32([kp1[m.trainIdx].pt for m in good])
H, inliers = cv2.findHomography(src, dst, cv2.RANSAC, 3.0)
```